Shadewater Labs Explainer

Webp Me Daddy

A recipe-driven image pipeline for turning messy website assets into production-ready WebP outputs with framing-aware crops, structured metadata, proof sheets, snippets, and codebase audits.

Shave bytes. Keep vibes. Give website images a real production loop.

Project Position

Shadewater Labs image workflow

CLI-first today, product-ready tomorrow

Built around proofing, audits, and shipping

Recipes	Command Routes	Scripts	Snippet Targets
9	**9**	**5**	**4**
Semantic asset roles built into the skill	Top-level CLI paths for still and animated assets	Helpers inside the local skill bundle	HTML, React, Next.js, and Astro outputs

Why It Exists

Most teams do not have an image pipeline. They have a pile of manual exports, inconsistent filenames, weak alt text, forgotten width and height attributes, and no proofing. Webp Me Daddy turns that chaos into a repeatable website workflow.

Semantic recipes

Prepare heroes, review art, blog covers, avatars, poster crops, and transparent logos with defaults that match the real slot instead of one-off image math.

Proof before ship

Generate surface proofs and batch boards so crop problems, matte halos, and contrast issues show up before they hit production.

Metadata with teeth

Use accessibility modes, visible-text inputs, and usage overrides so alt text and captions are deliberate instead of filler.

Audit and cleanup

Review a live codebase for legacy formats, missing markup, animated assets, unused originals, and safe autofix opportunities.

Core Workflow

The workflow is strongest when it follows the same operator loop every time: understand the placement, preview intentionally, prove the output visually, then clean up the codebase around it.

01

Audit the project

Scan public and src usage, surface stale formats, markup gaps, unused files, and animated assets that belong in the loop optimizer.

02

Dry-run the prep

Preview recipes, framing, responsive sizes, and lint status before writing anything to disk.

03

Proof the visuals

Review dark, light, and checker surfaces so transparent assets and crops get real QA instead of guesswork.

04

Ship snippets and fixes

Generate sidecars, manifests, framework snippets, fix plans, safe autofix patches, and cleanup output.

Best-Fit Use Cases

- Homepage hero banners that need the right crop, eager loading, and responsive WebP variants.
- Review-note and blog-cover art derived from posters without destroying the subject or readable title treatment.
- Transparent brand marks and wordmarks that need proofing on dark, light, and checker surfaces.
- Shared public assets that need usage-specific alt text across multiple placements.
- Codebase cleanup passes where image debt is slowing launches down.
- SEO handoff flows where Shadewater SEO Report identifies asset work and Webp Me Daddy handles the actual remediation.

What Comes Back

Output 1 Optimized WebP assets with responsive variants	**Output 2** Versioned sidecars and manifests
Output 3 Paste-ready snippets across frameworks	**Output 4** Audit, cleanup, and proof artifacts

Command Patterns

Prepare one transparent logo lockup

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare public/logo.png
--recipe logo-lockup --accessibility-mode logo --public-root public --write-sidecar
```

Preview a batch review pass

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py batch public
--recipe review-hero --dry-run --proof-contact-sheet batch-proof.png --manifest image-manifest.json
```

Audit and apply safe fixes

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py audit C:/path/to/project
--apply-autofix --json image-audit.json
```

Consume the SEO image handoff

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py seo-handoff
seo-image-handoff.json
--dry-run --json seo-image-apply-report.json
```

Current Limits

- The core path is intentionally WebP-first today; AVIF and multi-format delivery still belong to the next layer of work.
- The audit can emit fix plans and safe autofix patches, but a full orchestrated apply-everything workflow is still future product work.
- The operator surface is strong in CLI form, but the broader hosted app experience is still an upcoming phase.